

```
// api/generate-video.js
// Starts a video generation job on fal.ai
// and returns the exact status/result URLs
// that fal.ai itself provides (instead of
// guessing them) – this is the fix for
// the 405 errors, which happened because
// we were reconstructing URLs ourselves.

export const config = { runtime: 'edge' };

const MODELS = {
  kling: {
    image: 'fal-ai/kling-
video/v2.6/pro/image-to-video',
    text: 'fal-ai/kling-
video/v2.6/pro/text-to-video',
  },
  veo: {
    image: 'fal-ai/veo3/image-to-video',
    text: 'fal-ai/veo3',
  },
};

export default async function handler(req)
{
  if (req.method !== 'POST') {
    return json({ error: 'Method not
allowed' }, 405);
  }
}
```

```

    const key = process.env.FAL_KEY;
    if (!key) {
      return json({ error: 'Server not
configured: FAL_KEY missing.' }, 500);
    }

    let body;
    try {
      body = await req.json();
    } catch {
      return json({ error: 'Invalid request
body.' }, 400);
    }

    const { mode, engine, prompt,
imageDataUrl, imageDataUrls, images, aspect
} = body || {};
    const eng = engine === 'cinematic-pro' ?
'veo' : 'kling';
    const model = MODELS[eng] && MODELS[eng]
[model];
    if (!model) return json({ error: 'Unknown
mode/engine.' }, 400);

    const ratio =
['9:16', '1:1', '16:9'].includes(aspect) ?
aspect : '9:16';
    const defaultPrompt = prompt ||
'Cinematic real estate shot, slow smooth
camera motion, warm natural light, premium
look';

    // Preferred: images = [{dataUrl,
prompt}, ...] – each clip can have its own

```

```

description.
    // Fallback: imageDataUrls = [url, ...]
    (older format) or a single imageUrl,
    all sharing `prompt`.
    let imgEntries = [];
    if (Array.isArray(images) &&
    images.length) {
        imgEntries = images.map(i => ({
        imageUrl: i.imageUrl, prompt: (i.prompt &&
        i.prompt.trim()) || defaultPrompt }));
    } else if (Array.isArray(imageDataUrls)
    && imageDataUrls.length) {
        imgEntries = imageDataUrls.map(u => ({
        imageUrl: u, prompt: defaultPrompt }));
    } else if (imageUrl) {
        imgEntries = [{ imageUrl: imageUrl,
        prompt: defaultPrompt }];
    }

    const jobs = [];
    if (mode === 'image') {
        if (!imgEntries.length) return json({
        error: 'No image provided.' }, 400);
        for (const entry of imgEntries) {
            const input = { prompt: entry.prompt,
            duration: '5', aspect_ratio: ratio,
            generate_audio: true };
            if (eng === 'kling')
            input.start_image_url = entry.imageUrl; else
            input.image_url = entry.imageUrl;
            jobs.push(input);
        }
    } else {
        jobs.push({ prompt: defaultPrompt,

```

```

duration: '5', aspect_ratio: ratio,
generate_audio: true });
}

try {
  const started = [];
  for (const input of jobs) {
    const res = await
fetch(`https://queue.fal.run/${model}`, {
  method: 'POST',
  headers: { 'Authorization': `Key
${key}`, 'Content-Type': 'application/json'
},
  body: JSON.stringify(input),
});
    const data = await res.json();
    if (!res.ok) return json({ error:
formatFalError(data), raw: data },
res.status);

    // Use fal.ai's own URLs – do not
reconstruct them ourselves.
    started.push({
      request_id: data.request_id,
      status_url: data.status_url ||
`https://queue.fal.run/${model}/requests/${
data.request_id}/status`,
      response_url: data.response_url ||
`https://queue.fal.run/${model}/requests/${
data.request_id}`,
    });
  }
  return json({ jobs: started });
} catch (e) {

```

```

        return json({ error: 'Could not reach
fal.ai: ' + e.message }, 502);
    }
}

function formatFalError(data) {
    const d = data?.detail;
    if (!d) return data?.message || 'fal.ai
request failed.';
    if (typeof d === 'string') return d;
    if (Array.isArray(d)) {
        // FastAPI-style validation errors:
        [{loc:[...], msg:"...", type:"..."}]
        return d.map(e => {
            if (typeof e === 'string') return e;
            const field = Array.isArray(e?.loc) ?
e.loc.join('.') : '';
            return field ? `${field}: ${e?.msg ||
JSON.stringify(e)}` : (e?.msg ||
JSON.stringify(e));
        }).join(' | ');
    }
    if (typeof d === 'object') {
        try { return JSON.stringify(d); } catch
{ return 'fal.ai request failed.'; }
    }
    return String(d);
}

function json(obj, status = 200) {
    return new Response(JSON.stringify(obj),
{
    status,
    headers: { 'Content-Type':

```

```
'application/json' },  
  });  
}
```